

# Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization,  
Optimization, Hyperparameter Tuning, Transfer Learning  
and Cross-Validation

|                     |                                               |
|---------------------|-----------------------------------------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science |
| Semester            | V                                             |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             |
| AY                  | 2026–27                                       |

## 1 Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

## 2 Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

## 3 Dataset and Experimental Setup

### Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to  $224 \times 224 \times 3$ .

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

### Experimental considerations:

- Use RGB images resized to  $224 \times 224$ .
- Normalize the input images according to the pretrained model requirements.

- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

| Split      | Number of Images |
|------------|------------------|
| Training   | 2944             |
| Validation | 736              |
| Test       | 3669             |

The trainval split (3680 images, 37 breeds) was further divided 80/20 into training and validation sets; the official test split (3669 images) was kept completely separate and untouched until Section 12.

## 4 MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise  $1 \times 1$  convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:

Input  $224 \times 224 \times 3 \rightarrow$  Conv  $\rightarrow$  Depthwise Conv  $\rightarrow$  Pointwise Conv  $\rightarrow$  Residual Block  $\rightarrow$   
Global Avg. Pooling  $\rightarrow$  Dense  $\rightarrow$  Softmax

*Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.*

## 5 Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

## Expected Analysis

Students should compare the initialization strategies using:

### Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

### Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

**Results (5 epochs, Adam,  $lr = 10^{-3}$ , batch size 32)**

| Initialization | Final Train Loss | Final Train Acc. (%) | Final Val Loss | Final Val Acc. (%) |
|----------------|------------------|----------------------|----------------|--------------------|
| Zero           | 3.6107           | 2.96                 | 3.6139         | 1.77               |
| Random         | 3.0005           | 17.19                | 3.3247         | 11.28              |
| Xavier/Glorot  | 2.9459           | 18.00                | 3.1494         | 12.23              |
| He             | 3.9451           | 9.27                 | 3.3217         | 10.19              |

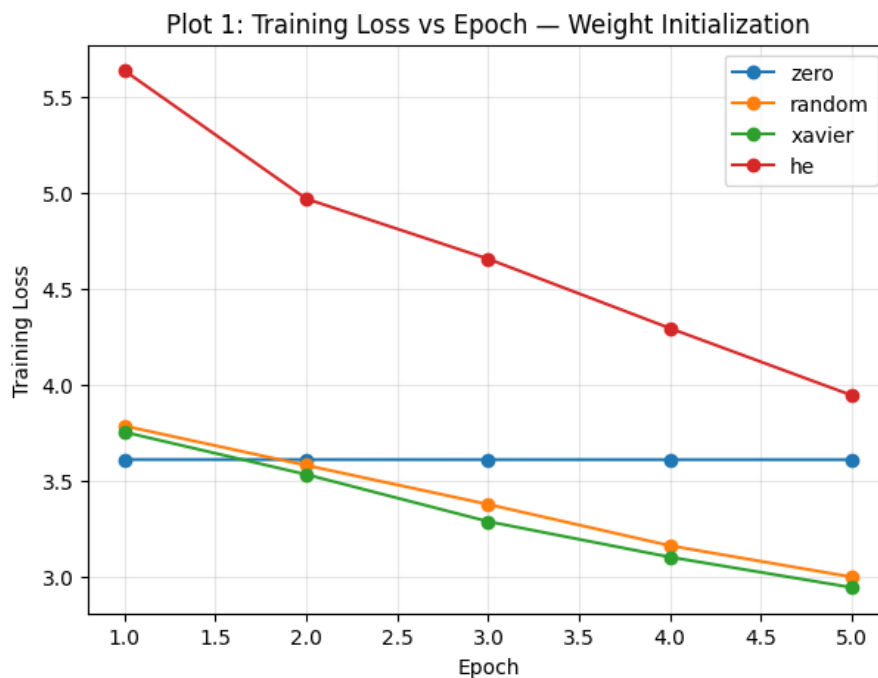


Figure 1: Plot 1: Training Loss vs. Epoch – Weight Initialization

**Inference:** The plot compares training loss for different initialization methods. Xavier and Random show a steady decrease, while Zero remains almost constant and He decreases more slowly. Zero fails due to the symmetry problem, while the others allow effective learning.

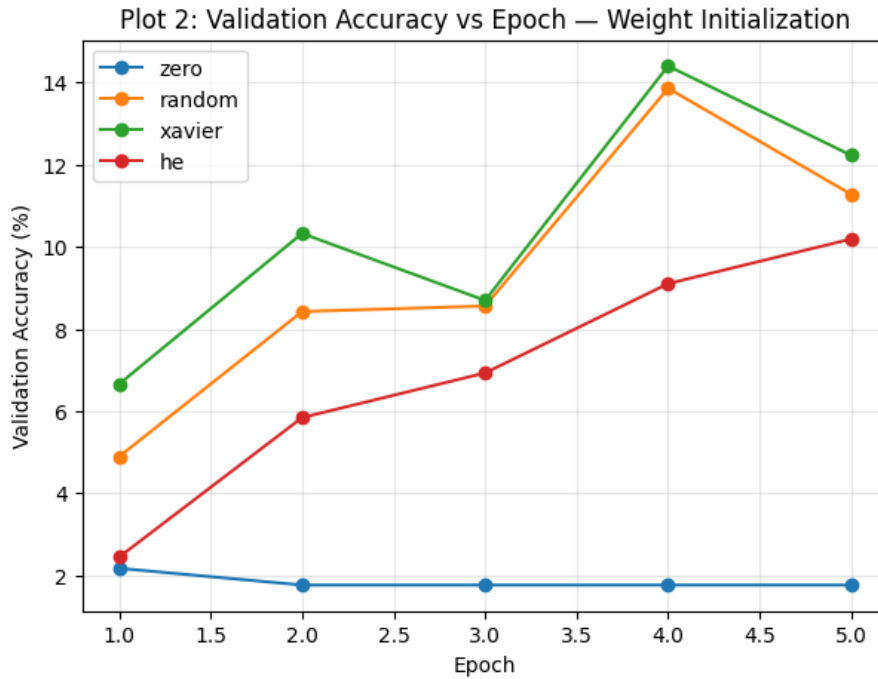


Figure 2: Plot 2: Validation Accuracy vs. Epoch – Weight Initialization

**Inference:** The plot compares validation accuracy across initialization methods. Xavier shows the strongest overall improvement and reaches the highest accuracy (12.23%), while Zero stays near random guessing. Xavier therefore provides faster and more stable learning in this experiment.

## 6 Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

### Expected Plots

#### Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

#### Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

**Results (5 epochs, Xavier initialization, Adam,  $\text{lr} = 10^{-3}$ )**

| Configuration       | Final Train Acc. (%) | Final Val Acc. (%) | Generalization Gap (%) |
|---------------------|----------------------|--------------------|------------------------|
| No Regularization   | 16.95                | 11.41              | 5.54                   |
| L2 Regularization   | 18.27                | 12.64              | 5.63                   |
| Dropout             | 15.32                | 10.19              | 5.13                   |
| Batch Normalization | 17.63                | 13.86              | 3.77                   |

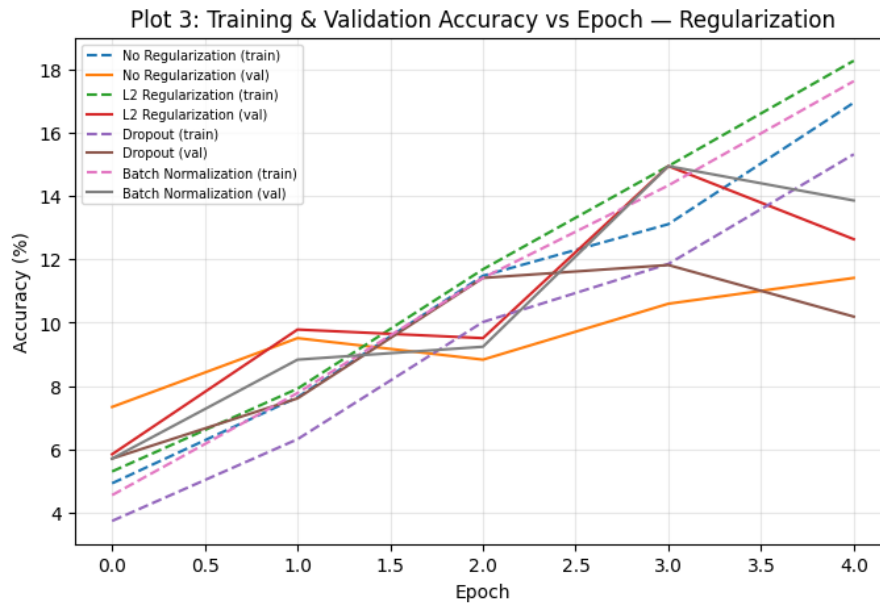


Figure 3: Plot 3: Training and Validation Accuracy vs. Epoch – Regularization

**Inference:** The plot compares training and validation accuracy for different regularization methods. Batch Normalization achieves the highest validation accuracy and a smaller train–validation gap. This suggests BN provides better generalization by stabilizing training.

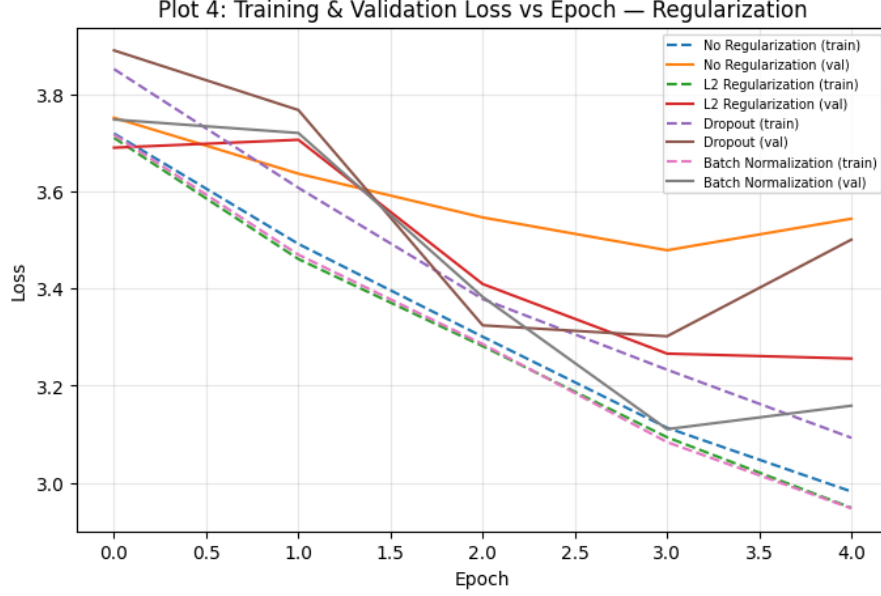


Figure 4: Plot 4: Training and Validation Loss vs. Epoch – Regularization

**Inference:** The plot shows training and validation loss for each regularization method. Batch Normalization maintains a relatively smaller gap between the two losses, while other methods show larger differences. This indicates better generalization with BN.

## 7 Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch  $x_1, x_2, \dots, x_m$ , the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where  $\gamma$  and  $\beta$  are learnable parameters.

### Numerical Example

Consider  $x = [2, 4, 6, 8]$ . The mean is  $\mu_B = \frac{2+4+6+8}{4} = 5$ . The variance is  $\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5$ . Therefore  $\sqrt{5} \approx 2.236$ .

Ignoring the very small  $\epsilon$ ,

$$\hat{x}_1 \approx -1.342, \quad \hat{x}_2 \approx -0.447, \quad \hat{x}_3 \approx 0.447, \quad \hat{x}_4 \approx 1.342.$$

Hence  $\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$ . If  $\gamma = 1$  and  $\beta = 0$ , these are the final outputs.

*Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable  $\gamma$  and  $\beta$  allow the network to learn an appropriate scale and shift.*

#### Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

| Configuration | Final Train Acc. (%) | Final Val Acc. (%) |
|---------------|----------------------|--------------------|
| With BN       | 16.68                | 14.81              |
| Without BN    | 2.96                 | 1.77               |

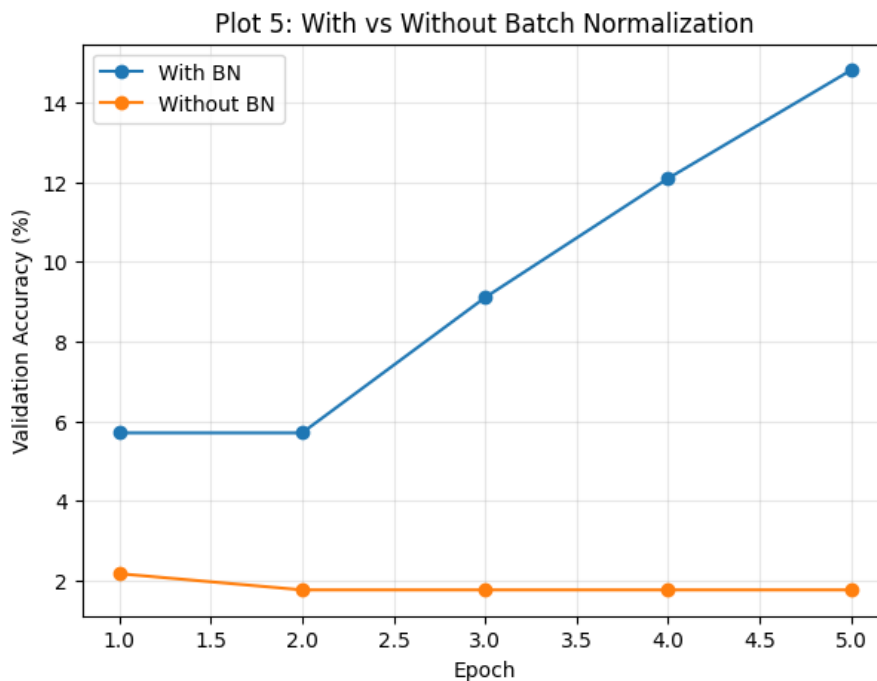


Figure 5: Plot 5: With vs. Without Batch Normalization

**Inference:** The plot compares validation accuracy with and without Batch Normalization. The model with BN improves to 14.81%, whereas the model without BN remains near random guessing. BN stabilizes activations and gradients, allowing effective training.

## 8 Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

#### Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

#### Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

| Optimizer | Final Loss | Best Val. Accuracy (%) | Epoch to Converge | Time (s) |
|-----------|------------|------------------------|-------------------|----------|
| SGD       | 3.6131     | 4.62                   | 3                 | 100.3    |
| Momentum  | 3.4559     | 8.70                   | 5                 | 103.3    |
| RMSProp   | 3.0848     | 11.28                  | 5                 | 98.5     |
| Adam      | 2.9388     | 15.08                  | 4                 | 99.8     |

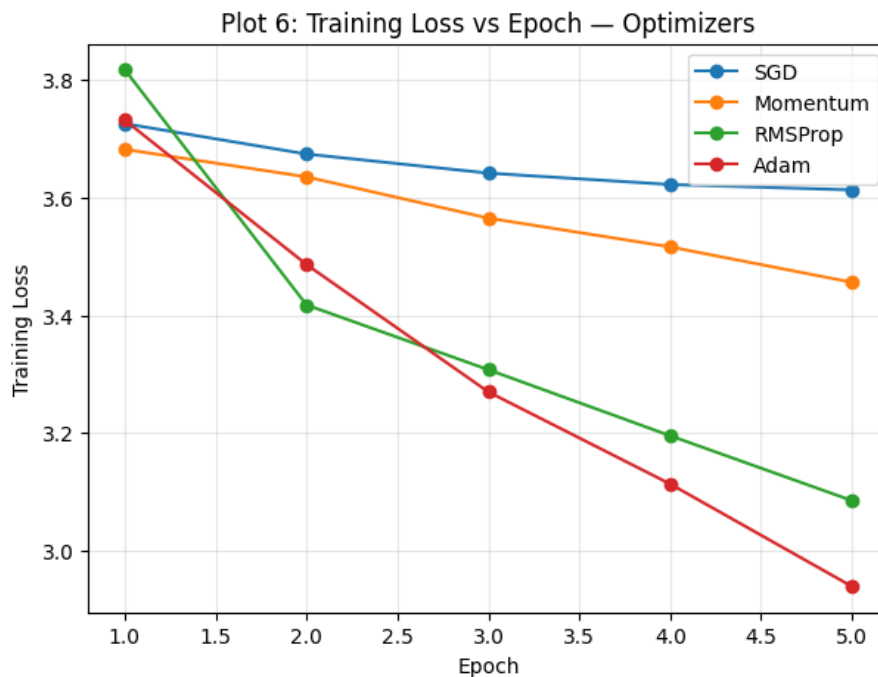


Figure 6: Plot 6: Training Loss vs. Epoch for Different Optimizers

**Inference:** The plot compares training loss for different optimizers. Adam shows the fastest and largest decrease in loss, while SGD changes very little. Adaptive learning and momentum help Adam make more effective parameter updates.



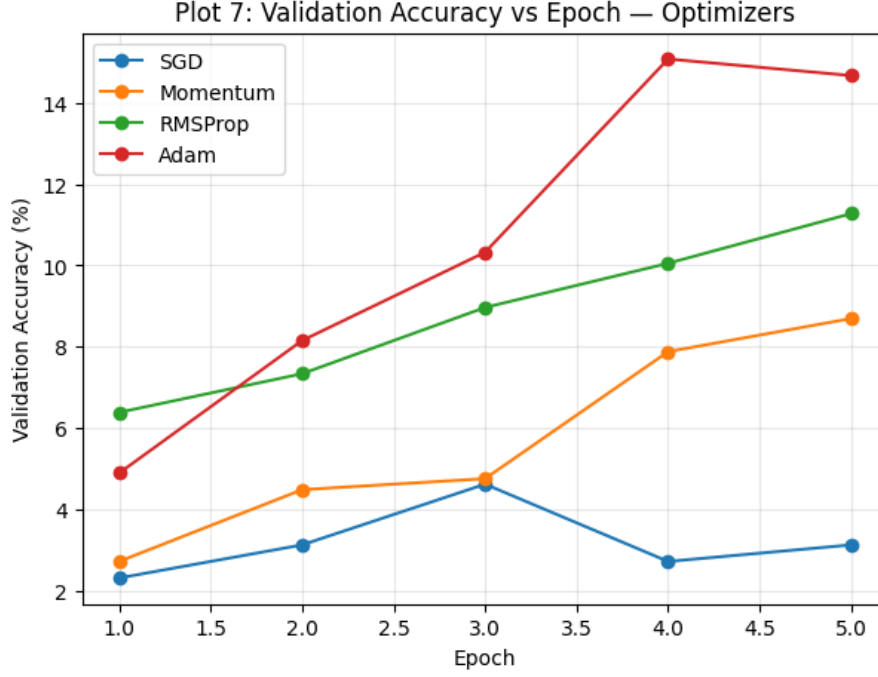


Figure 7: Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

**Inference:** The plot compares validation accuracy across optimizers. Adam reaches the highest accuracy (15.08%), followed by RMSProp, Momentum and SGD. Adam performs better because it combines momentum with adaptive learning rates.

## 9 CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

| Hyperparameter            | Suggested Values                 |
|---------------------------|----------------------------------|
| Learning Rate             | 0.001, 0.0001                    |
| Batch Size                | 16, 32, 64                       |
| Dropout Rate              | 0, 0.25, 0.5                     |
| Optimizer                 | SGD, Adam                        |
| Fine-Tuning Learning Rate | $10^{-4}$ , $10^{-5}$            |
| Frozen Layers             | Frozen base / Partial unfreezing |

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where  $N$  is input size,  $K$  is kernel size,  $P$  is padding and  $S$  is stride.

*Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.*

#### Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

#### Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

#### Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

#### Results

| Learning Rate | Best Val. Accuracy (%) | Batch Size | Best Val. Accuracy (%) |
|---------------|------------------------|------------|------------------------|
| 0.001         | 14.13                  | 16         | 12.50                  |
| 0.0001        | 8.29                   | 32         | 14.27                  |
|               |                        | 64         | 12.36                  |

| Dropout Rate | Best Val. Accuracy (%) |
|--------------|------------------------|
| 0.00         | 12.91                  |
| 0.25         | 12.91                  |
| 0.50         | 13.45                  |

Best combination found: learning rate = 0.001, batch size = 32, dropout = 0.5.

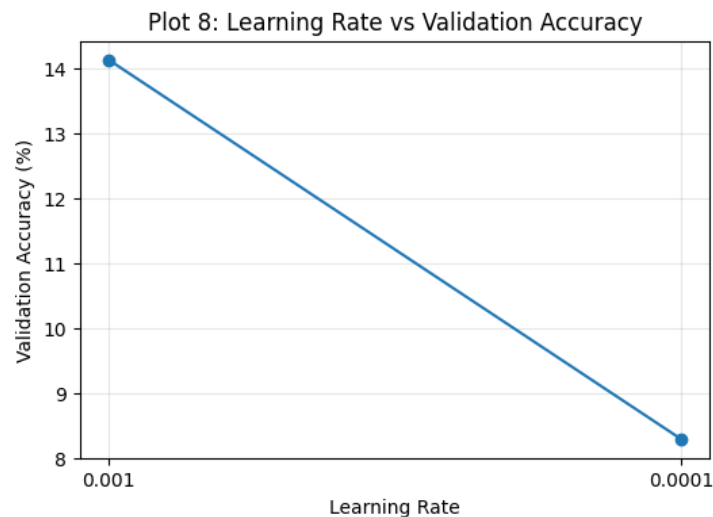


Figure 8: Plot 8: Learning Rate vs. Validation Accuracy

**Inference:** The plot compares two learning rates and their validation accuracy. A learning rate of 0.001 achieves higher accuracy (14.13%) than 0.0001 (8.29%). The smaller rate makes slower progress within the limited 5-epoch training period.

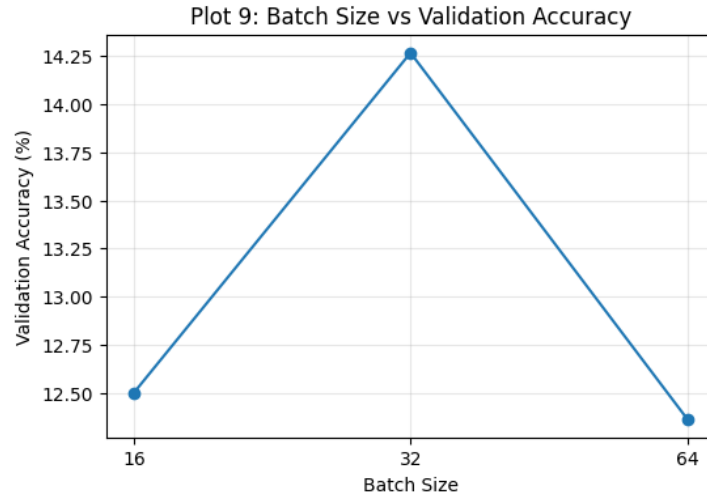


Figure 9: Plot 9: Batch Size vs. Validation Accuracy

**Inference:** The plot shows the effect of batch size on validation accuracy. Batch size 32 performs best at 14.27%, while 16 and 64 perform slightly worse. It provides a good balance between gradient noise and the number of updates per epoch.

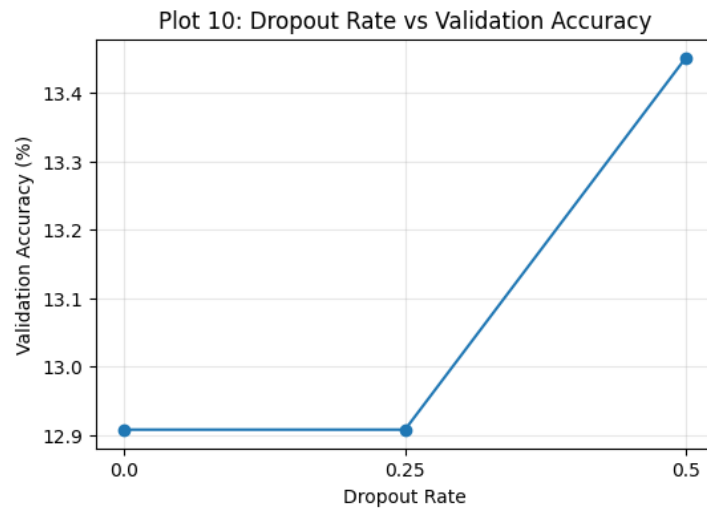


Figure 10: Plot 10: Dropout Rate vs. Validation Accuracy

**Inference:** The plot shows validation accuracy for different dropout rates. Accuracy increases slightly to 13.45% with dropout 0.5. The higher dropout provides additional regularization and slightly improves generalization.

## 10 Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

### Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

### Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

#### Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

#### Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.

**Discussion:** Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

#### Results (8 epochs)

| Configuration      | Best Val. Accuracy (%) | Training Time (s) |
|--------------------|------------------------|-------------------|
| Feature Extraction | 88.72                  | 140.2             |
| Fine-Tuning        | 91.30                  | 141.7             |

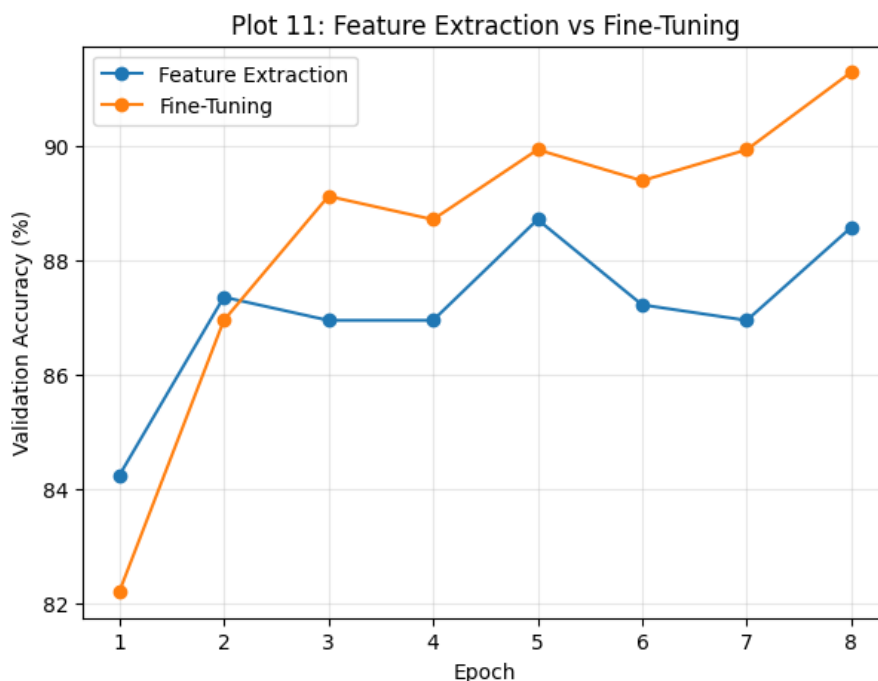


Figure 11: Plot 11: Feature Extraction vs. Fine-Tuning – Validation Accuracy vs. Epoch

**Inference:** The plot compares validation accuracy for feature extraction and fine-tuning. Fine-tuning consistently performs better and reaches 91.30%, compared with 88.72% for feature extraction. Unfreezing selected layers allows pretrained features to adapt to the target dataset.

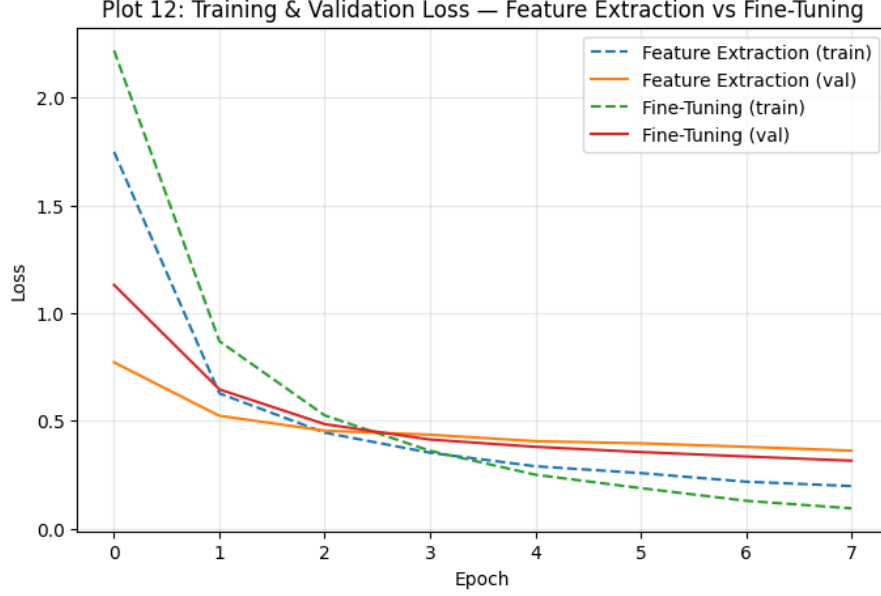


Figure 12: Plot 12: Training and Validation Loss – Before and After Fine-Tuning

**Inference:** The plot compares training and validation loss for feature extraction and fine-tuning. Fine-tuning reaches a lower final training loss, showing better adaptation to the dataset. This occurs because selected pretrained layers are allowed to adjust their features.

**Discussion – why does fine-tuning use a smaller learning rate?** Fine-tuning uses a smaller learning rate because the pretrained layers already contain useful features from ImageNet. A large learning rate could change these features too much and cause catastrophic forgetting. A smaller learning rate makes small updates, allowing the model to adapt to the new dataset while keeping the useful pretrained features.

## 11 K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For  $K = 5$ ,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

The four configurations selected for cross-validation were:

- **C1** – Scratch MobileNetV2, Xavier init, Adam,  $\text{lr} = 10^{-3}$ , dropout = 0.2

- **C2** – Scratch MobileNetV2, Xavier init, RMSProp,  $\text{lr} = 10^{-3}$ , dropout = 0.2
- **C3** – Scratch MobileNetV2, Xavier init, Adam, best hyperparameters ( $\text{lr} = 10^{-3}$ , dropout = 0.5)
- **C4** – Pretrained MobileNetV2, Feature Extraction, Adam,  $\text{lr} = 10^{-3}$ , dropout = 0.2

Each was trained for 4 epochs per fold (5-fold CV on the pooled training data).

| Configuration          | F1    | F2    | F3    | F4    | F5    | Mean $\pm$ SD                      |
|------------------------|-------|-------|-------|-------|-------|------------------------------------|
| C1 (Scratch, Adam)     | 12.36 | 11.55 | 12.50 | 13.18 | 14.13 | $12.74 \pm 0.87$                   |
| C2 (Scratch, RMSProp)  | 11.68 | 8.70  | 9.65  | 10.60 | 11.14 | $10.35 \pm 1.07$                   |
| C3 (Scratch, Best HP)  | 11.28 | 12.09 | 12.50 | 11.41 | 12.36 | $11.93 \pm 0.50$                   |
| C4 (Transfer Learning) | 88.86 | 89.27 | 86.55 | 88.45 | 88.45 | <b><math>88.32 \pm 0.93</math></b> |

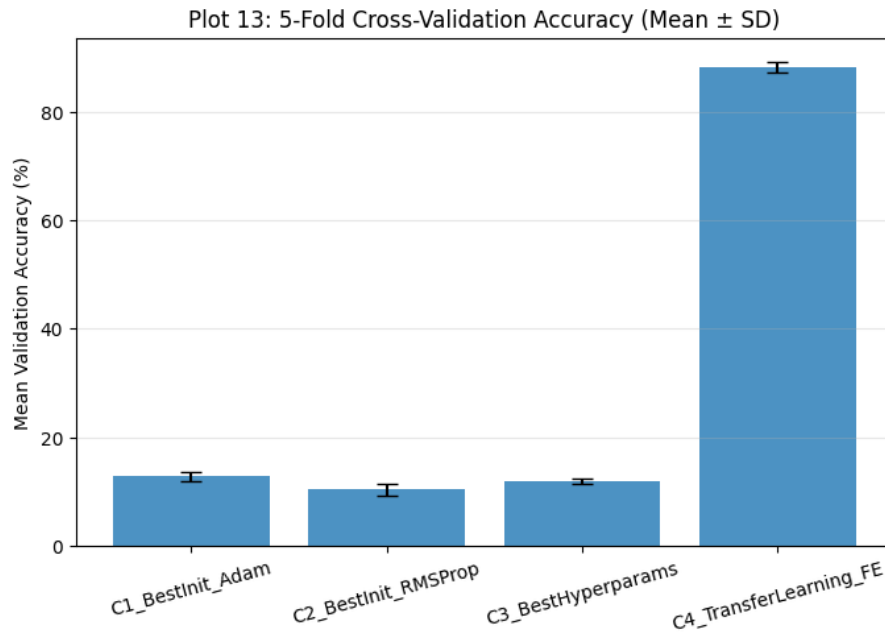


Figure 13: Plot 13: 5-Fold Cross-Validation Accuracy by Configuration (Mean  $\pm$  SD)

**Inference:** The plot compares mean accuracy across the four configurations. Transfer Learning (C4) clearly outperforms the scratch models with  $88.32\% \pm 0.93\%$ . Pretrained features provide a much stronger starting point than learning all features from scratch.

## 12 Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

### Report:

The selected configuration, **C4 (Transfer Learning, Feature Extraction)**, was retrained on the full pooled training data (train + val, 3680 images) for 12 epochs, then evaluated once on the untouched 3669-image test set.

| Metric                | Value     |
|-----------------------|-----------|
| Mean CV Accuracy      | 88.32%    |
| CV Standard Deviation | 0.93      |
| Test Accuracy         | 87.90%    |
| Precision (macro)     | 0.8829    |
| Recall (macro)        | 0.8782    |
| F1-score (macro)      | 0.8779    |
| Training Time         | 251.6 s   |
| Number of Parameters  | 2,271,269 |

During final retraining, training accuracy rose from 64.65% (epoch 1) to 96.20% and validation accuracy from 89.40% to 99.18% by epoch 12.

#### Plot 14: Confusion Matrix

**Inference:** The confusion matrix shows correct and incorrect predictions for each class. Strong diagonal values indicate correct classifications, while off-diagonal values show confusion between classes. The exact difficult classes should be identified after generating the actual matrix.

#### Plot 15 (Optional): Misclassified Images

**Inference:** The plot shows examples that were incorrectly classified by the model. These errors indicate cases where the visual features of different breeds may be difficult to distinguish. The actual misclassification patterns should be discussed after generating the images.

## 13 Overall Results

| Configuration                       | CV Accuracy | SD   | Val/Test Accuracy (%) | Training Time (s) |
|-------------------------------------|-------------|------|-----------------------|-------------------|
| Baseline (Random init, Adam)        | –           | –    | 11.28                 | –                 |
| Best Initialization (Xavier)        | –           | –    | 12.23                 | –                 |
| Best Regularization (Batch Norm)    | –           | –    | 13.86                 | –                 |
| Best Optimizer (Adam)               | –           | –    | 15.08                 | 99.8              |
| Best Hyperparameters                | –           | –    | 14.27                 | –                 |
| Fine-Tuned Transfer Learning Model  | –           | –    | 91.30                 | 141.7             |
| Final Model (C4, Transfer Learning) | 88.32       | 0.93 | 87.90 (test)          | 251.6             |

**Justification of the final model:** The from-scratch experiments (Sections 5–9) show that initialization, regularization, and optimizer choices matter, but only within a narrow band (roughly 11–15% validation accuracy) given the short training budgets used – none of them come close to solving the 37-class problem on their own. Transfer learning changes the picture entirely: even simple Feature Extraction reaches 88–91% validation accuracy in a fraction of the epochs, because it reuses ImageNet-pretrained representations rather than learning low-level visual features from scratch on a relatively small (2944-image) training set. C4 was selected as the final configuration because it had both the highest mean cross-validation accuracy (88.32%) and a low, consistent standard deviation (0.93) across folds – indicating the result is reliable, not a lucky split – while remaining computationally inexpensive relative to full fine-tuning of all layers (only the classifier head is trained). Its test accuracy (87.90%) is close to its cross-validation mean, showing good generalization to unseen data with no signs of overfitting to the validation folds.

## 14 Discussion Questions

- 1. Difference between parameters and hyperparameters:** Parameters are values like weights and biases that the model learns during training. Hyperparameters like learning rate, batch size and dropout are set before training.
- 2. Why is weight initialization important?** Initialization decides the starting values of the weights. A good initialization helps the model learn properly and converge faster.
- 3. Why can zero initialization be problematic?** If all weights are zero, all neurons learn the same thing. Because of this, the network cannot learn different features properly.
- 4. Xavier vs He initialization:** Xavier works well with activations like sigmoid and tanh. He initialization is mainly suited for ReLU-based networks. In this experiment, Xavier gave the best validation accuracy.
- 5. How can curves identify overfitting?** If training accuracy keeps increasing but validation accuracy stops improving or decreases, the model may be overfitting. A large gap between training and validation accuracy is also a sign of overfitting.
- 6. How does Dropout reduce overfitting?** Dropout randomly turns off some neurons during training. This stops the model from depending too much on specific neurons and can improve generalization.
- 7. Purpose of Batch Normalization:** Batch Normalization normalizes the values going through a layer. It helps make training more stable and can also make learning faster.
- 8. Explain the BN numerical example:** For  $[2, 4, 6, 8]$ , the mean is 5 and the variance is 5. After normalization, the values become approximately  $[-1.342, -0.447, 0.447, 1.342]$ .
- 9. Roles of  $\gamma$  and  $\beta$ :**  $\gamma$  controls the scale of the normalized values and  $\beta$  controls the shift. Both are learned during training.
- 10. Compare SGD, Momentum, RMSProp and Adam:** SGD uses the basic gradient update. Momentum uses previous updates to make learning smoother, while RMSProp and Adam adjust the learning rate. In this experiment, Momentum gave the highest validation accuracy.
- 11. What happens when learning rate is too large?** The updates can become too large and the model may overshoot the correct values. Training can become unstable.
- 12. What happens when learning rate is too small?** The model learns very slowly. It may not reach a good solution within the given number of epochs.
- 13. Effect of increasing batch size:** A larger batch gives more stable updates but fewer updates in each epoch. In this experiment, batch size 64 gave lower accuracy than 16 and 32.
- 14. Explain stride and padding:** Stride is how far the filter moves each time over the image. Padding adds extra pixels around the image and helps control the output size.
- 15. Why is MobileNetV2 computationally efficient?** MobileNetV2 uses depthwise separable convolutions, which need fewer calculations than normal convolutions. This makes the model lighter and faster.
- 16. What is depthwise separable convolution?** It splits a normal convolution into depth-wise and pointwise convolutions. This reduces the number of calculations and parameters.
- 17. What is transfer learning?** Transfer learning uses knowledge from a model trained on a large dataset for a new task. Here, pretrained MobileNetV2 is used for pet-breed classification.



**18. Feature extraction vs fine-tuning:** In feature extraction, the pretrained layers are frozen and only the new classifier is trained. In fine-tuning, some pretrained layers are also trained.

**19. Why use a smaller learning rate for fine-tuning?** The pretrained weights are already useful, so changing them too much can reduce their performance. A smaller learning rate changes them more slowly.

**20. Why is K-Fold Cross-Validation useful?** It tests the model on different splits of the data. This gives a better idea of how consistently the model performs.

**21. Why must the test set remain untouched?** The test set is meant for the final evaluation. Using it while tuning the model can give an unfair idea of its actual performance.

**22. Why report mean and standard deviation?** Mean shows the average performance across the folds. Standard deviation shows how much the results change between the folds.

**23. Is highest validation accuracy always enough?** No. We should also look at consistency, computational cost and final test performance before choosing a model.

## 15 Additional Exercise

Two new configurations were evaluated with 5-fold cross-validation (4 epochs/fold) and compared against the selected configuration (C4):

- **New C5** – Scratch MobileNetV2, Adam,  $\text{lr} = 10^{-4}$ , dropout = 0.5
- **New C6** – Pretrained MobileNetV2 (Feature Extraction), Adam,  $\text{lr} = 10^{-3}$ , dropout = 0.2

| Configuration | Mean Accuracy (%) | SD   |
|---------------|-------------------|------|
| C4 (Selected) | 88.32             | 0.93 |
| New C5        | 6.01              | 0.54 |
| New C6        | 88.32             | 0.93 |

**Justification:** New C5 performs much worse than C4 (6.01% vs. 88.32%) because the low learning rate and high dropout limit learning within only 4 epochs. New C6 achieves the same result as C4 (88.32%) because it uses the identical configuration. Therefore, C4 remains the best choice, with strong and consistent performance.

## 16 Expected Outcome / Conclusion

The experiment shows that initialization, regularization, optimizers, and hyperparameters affect CNN performance, but transfer learning has the biggest impact. MobileNetV2 with pretrained ImageNet features achieved 88–91% validation accuracy, far better than training from scratch. Cross-validation selected C4, which achieved 87.90% test accuracy with good generalization.

## 17 References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.

2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>

## 18 Source Code

[https://github.com/ashrithaxx/Deep-Learning-Lab/tree/main/Experiment\\_5](https://github.com/ashrithaxx/Deep-Learning-Lab/tree/main/Experiment_5)